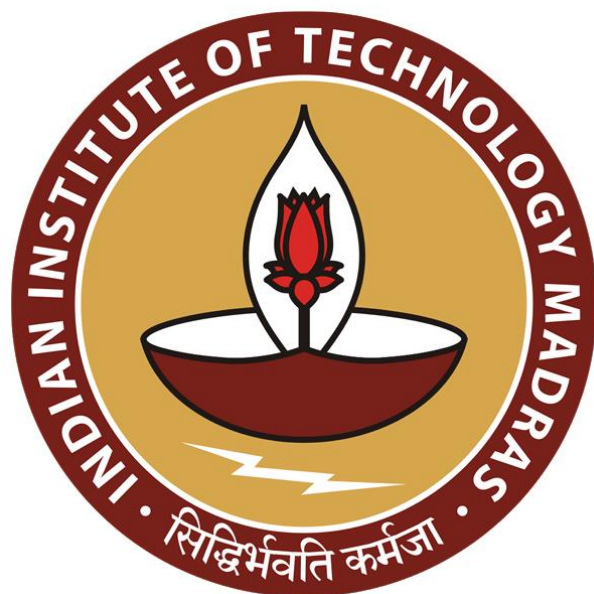




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Reflection

So, let us look at a feature of Java that is called Reflection.

(Refer Slide Time: 00:20)

Reflection

Wikipedia
Reflective programming or reflection is the ability of a process to examine, introspect, and modify its own structure and behaviour.

- Two components involved in reflection
 - **Introspection**
A program can observe, and therefore reason about its own state.
 - **Intercession**
A program can modify its execution state or alter its own interpretation or meaning.

Madhavan Mukund

So, Reflection in the context of programming languages talks about the ability of a language to examine itself. So, we want to be able, within the language, within the programs that we write in that language, to be able to introspect, to be able to examine what is happening inside the code so we can talk about the code from within the code more or less. So, that is what reflection means.

So, there are two parts to reflection normally. So, the first part is to understand what is happening. So, you might want to check at runtime what is the type of a variable or you want to check something of that kind where the two variables have the same type. The other thing is to intervene. So, you want to intercede, as it is called, intercession, so you might want to actually change something. If something is of some type then change its type.

So, these are the two things that you typically would like to do if you have a reflective capability in your programming language. Find out what is going on, and based on that take some kind of,

so this action of course, all programs do something at run time based on the values. So, this is not something as trivial as updating a value of a variable. It is more like changing the type of a variable or creating a new object or something like that.

So, that is what we mean by interceding. It is not just something as simple as updating the value which is what any way a normal piece of code will do all the time. Based on the current flow of the thing you might want to update a variable one way or another way so, that is not reflection. Reflection is really talking about some kind of structural change in the program that you perform as a result of what the program has done so far.

(Refer Slide Time: 1:59)

IIT Madras
BSc Degree

Reflection in Java

- Simple example of introspection

```
Employee e = new Manager(...);  
...  
if (e instanceof Manager){  
    ...  
}
```
- What if we don't know the type that we want to check in advance?
- Suppose we want to write a function to check if two different objects are both instances of the same class?

```
public static boolean classequal(Object o1, Object o2){  
    ...  
    // return true iff o1 and o2 point to objects of same type  
    ...  
}
```

Madhavan Mukund

Reflection

Programming Concepts using Java 3/14

So, the simplest kind of reflection that we see in Java which we have also seen in languages like Python is you can examine types. So, you have passed a parameter and you want to find out at some point what exactly the parameter is, and especially in a context of a language like Java where we have this type hierarchy and we have this compatibility of types across this hierarchy, so we might at any point get something which is notionally of a super type but is actually of a sub type.

And then, we might want to ask if it is of the sub type whether we can perform additional operations on it because the subtype will have more capabilities, but we do not want to do this unless we know for sure that the value that we have is of the subtype. So, in this case we can use

this Java instance off to validate that the so called Employee e which has been declared to be of a super type is actually at run time, something more sophisticated, namely a Manager.

Now, this is useful if we know in advance what we are checking. So, when we write the code we have some fixed types like Manager and Employee and we want to know whether something which was declared to be an Employee is actually a Manager or not. But what if we do not know this? What if we generally want to check something about two arbitrary objects that are given to us.

So, supposing we want to do something as simple as validate whether two parameters that are passed to a function are both objects of the same class. So, we want to write a function class equal which will take two arbitrary objects and check whether or not they are the same class. So, this is now written in this style where I am not using a generic programming thing, I am using the class hierarchy.

So, any parameter because it is compatible with object can be passed for o1 and o2, any parameter, so I can pass two dates, I can pass a date and an Employee, I can pass a Manager and an Employee, and then depending on whether they are equal or not I must say true or false.

(Refer Slide Time: 3:58)



Reflection in Java ...

```
public static boolean classequal(Object o1, Object o2){...}
```

- Can't use `instanceof`
 - Will have to check across all defined classes
 - This is not even a fixed set!
- Can't use generic type variables
 - The following code is syntactically disallowed

```
if (o1 instanceof T) { ...}
```

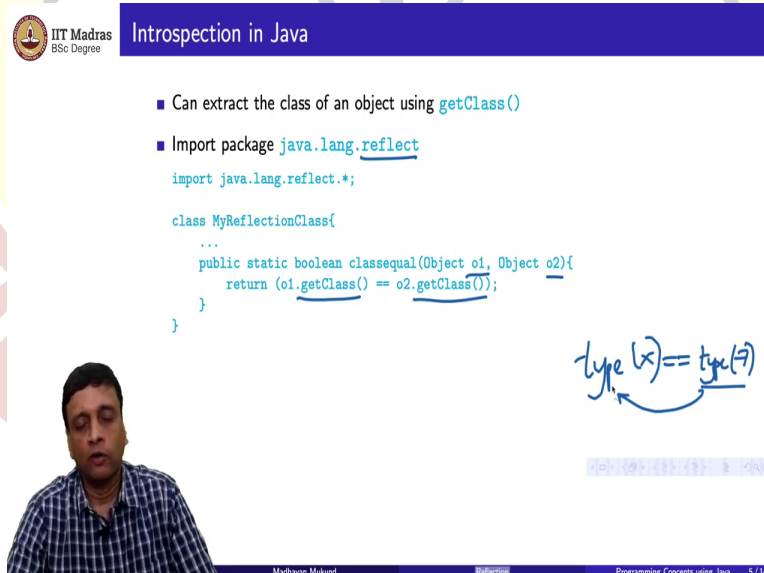


So, how would one go about this? Well we cannot exactly use instanceof, because we do not know, unlike the earlier case where we knew that we were checking something was an instance of Manager. Here we really do not know what we are checking for. We just know that o1 has a type, o2 has a type, and we want to check whether o1 and o2 have the same type.

So, o1 is an instance of some type, o2 is also an instance of the same type but we have to run over all possible types and we cannot possibly know that because there are all the types that we are using but we are of course importing and inheriting a lot of types which come because we are part of this Java tree under object. So, the list of all possible classes and types which are compatible with object is not even known to us. So, we cannot even find out easily what we must take. So, this is clearly infeasible.

On the other hand, this kind of a test is not allowed. We are not allowed in Java to check whether something compare it with a type variable. So, we will see this, but for generic types this instanceof is not well defined. So, we cannot use generic types to get around this. So, what can we do?

(Refer Slide Time: 5:06)



IIT Madras
BSc Degree

Introspection in Java

- Can extract the class of an object using `getClass()`
- Import package `java.lang.reflect`

```
import java.lang.reflect.*;

class MyReflectionClass{
    ...
    public static boolean classequal(Object o1, Object o2){
        return (o1.getClass() == o2.getClass());
    }
}
```

type(x) == type(y)

Madhavan Mahand Reflection Programming Concepts using Java 5/14

So, it turns out that Java provides this introspection, this reflective capability to ask questions about various things. So, one of the things that we can ask is what is the class that an object

belongs to. So, there is a function called `getClass()`. And where is this `getClass()`? It is in the package called `reflect`, so remember that we are talking about reflection.

So, Java has a `reflect` package. So, if we import this package then we have access to these functions which allow us to examine classes and do much more. So, at the very least what we can do is we can take `o1` and `o2`, and we can get the class of each, so I can say `o1.getClass()` and `o2.getClass()` and compare them.

If you remember, in Python, we had this `type`, we can say `type of x`, and we can say something like if `type of x` is equal to, say `type of 7` this would tell us whether or not `x` is an `int` or not. We can compare two types. It is not so easy to, I mean, you need to know the syntax to know what to compare it to as a context, as a constant, but if you have another value of a known type so you can just take an integer expression and its type will be of type `int`. You can take that type and you can compare it.

So this, Java can do with this `getClass()`, but this in Python, notice that this `type` thing is basically used for these built in types, whereas `getClass()` now tells us some how to do it for user defined classes in Java.

(Refer Slide Time: 6:32)

Introspection in Java

- Can extract the class of an object using `getClass()`
- Import package `java.lang.reflect`

```
import java.lang.reflect.*;
```

```
class MyReflectionClass{
    ...
    public static boolean classequal(Object o1, Object o2){
        return (o1.getClass() == o2.getClass());
    }
}
```

- What does `getClass()` return?

Mathavan Mahand Reflection Programming Concepts using Java 5/14

- Can extract the class of an object using `getClass()`

- Import package `java.lang.reflect`

```
import java.lang.reflect.*;

class MyReflectionClass{
    ...
    public static boolean classequal(Object o1, Object o2){
        return (o1.getClass() == o2.getClass());
    }
}
```

o1 = o2

- What does `getClass()` return?

- An object of type `Class` that encodes class information



Navigation icons: back, forward, search, etc.

Madhavan Mukund

Reflection

Programming Concepts using Java 5/14

So, what does `getClass()` actually give us? When I am saying `(o1.getClass() == o2.getClass())` what am I comparing? Is this the name of the class? Because that is what Python gives us. Python gives us a kind of a string which tells us something about the name of the class. So, it turns out that the Java `getClass()` gives us a much richer structure.

See what Java's `getClass()` gives us is another object, and again this is somewhat confusing but Java uses a built-in type for a class with a capital C to denote the type of this object. So, remember that a class object with a capital O is this root of the hierarchy and now a class with a capital C denotes generic information about classes. So, I can find out information about classes in objects of this type.

So, what we are getting here is a class, what we are getting here is a class, both of these are giving us classes and now we are asking whether this class is equal to this class or not.

सिद्धिर्भवति कर्मजा

(Refer Slide Time: 7:35)



The class `Class`

- A version of `classsequal` the explicitly uses this fact

```
import java.lang.reflect.*;

class MyReflectionClass{
    ...
    public static boolean classequal(Object o1, Object o2){
        Class c1, c2;
        c1 = o1.getClass();
        c2 = o2.getClass();
        return (c1 == c2);
    }
}
```



Madhavan Mukund

Reflection

Programming Concepts using Java 6/14



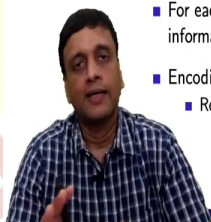
The class `Class`

- A version of `classsequal` the explicitly uses this fact

```
import java.lang.reflect.*;

class MyReflectionClass{
    ...
    public static boolean classequal(Object o1, Object o2){
        Class c1, c2;
        c1 = o1.getClass();
        c2 = o2.getClass();
        return (c1 == c2);
    }
}
```

- For each currently loaded class `C`, Java creates an object of type `Class` with information about `C`
- Encoding execution state as data — reification
 - Representing an abstract idea in a concrete form



Madhavan Mukund

Reflection

Programming Concepts using Java 6/14

So, we can actually, so in this case we have actually used this directly as an equality, but we can actually define two variables `c1` and `c2` which are of this Java class type and instantiate them with the two classes of the relative objects `o1` and `o2` passed to this and then check whether those two classes that get assigned.

So, this is just an elaborate version of the same thing earlier, except earlier we were not aware what that “`==`” is actually checking whereas here we are kind of aware now that both of them are actually class objects. We are checking that they are the same class objects. So, remember

this equality is going to be the reference equality, it is going to say that there is only one class object denoting these two and its the same class.

So, Java actually does this implicitly. So, for every class which has been loaded, so when a class is invoked as your Java compiler, as your Java code runs, it gets loaded into the runtime system. So, every class which is currently loaded has a representation of its properties in an object of this type. So, it is readily available to Java.

So, this is a way of taking some abstract information about the program, namely the class associated with different objects in the runtime code, and making them themselves into objects. So, you can think about the kind of a recursive thing going on here an object has a class and this class is represented in another object, and of course that object has a class but then it is the same class and it stops. So, we do not get into an infinite descend.

But the main point is that you are taking something which is kind of abstract, which is the property of the variables that you are using in your code and you are representing them as a concrete thing that you can examine. So, this is a kind of fancy word for it, reification. So, reification basically means representing something abstract as something concrete. So, basically, taking something and then putting it down in a place where you can actually examine.

(Refer Slide Time: 9:42)



Using the `Class` object

- Can create new instances of a class at runtime

```
...  
Class c = obj.getClass();  
Object o = c.newInstance();  
// Create a new object of same type as obj  
...
```



Using the `Class` object

- Can create new instances of a class at runtime

```
...
Class c = obj.getClass();
Object o = c.newInstance();
// Create a new object of same type as obj
...
```

- Can also get hold of the class object using the name of the class

```
...
String s = "Manager";
Class c = Class.forName(s);
Object o = c.newInstance();
...
```



Mathavan Mukund

Reflection

Programming Concepts using Java 7/14

Using the `Class` object

- Can create new instances of a class at runtime

```
...
Class c = obj.getClass();
Object o = c.newInstance();
// Create a new object of same type as obj
...
```

- Can also get hold of the class object using the name of the class

```
...
String s = "Manager";
Class c = Class.forName(s);
Object o = c.newInstance();
...
```

- ..., or, more compactly

```
...
Object o = Class.forName("Manager").newInstance();
// Man
```



Mathavan Mukund

Reflection

Programming Concepts using Java 7/14

So, now that we have this class object, we can do much more than just check whether two classes are equal. So, once we have this class object, at runtime we can actually create new objects of a given class. So, this is what we talked about in the beginning. So, there are two types of things that you might want to do in a reflection kind of mode, in a programming language.

One is to examine things about what is going on. So, that is what `getClass()` does. It allows us to examine at a given point what are the properties of a given object in terms of what class it belongs to. But now, what we are saying is that depending on that we can actually launch a new object of a given class, a class which we have never seen before. So we inherit a class maybe as a

parameter in a function and we can throw out an object of that class without ever having seen that class before.

So, there is this new instance function which allows me to take a C and given, and of course, since I do not know in some sense what that class is, this new instance I have to make it of a generic type object because I do not know what the class is, where it sits in the hierarchy. And then whoever is going to use it presumably will know what is going on and they will be able to change it.

So, we can also create such an object by knowing the name of a class. So, for instance if we know that there is a Manager class, then we can take the Manager name and create a class out of it, and then we can create an instance of that class. So, we can collapse this saying that we take the name, create a class for that name because we know that there is a class defined called Manager, and then we make an object of that type.

But remember that we cannot use this string here because this is a runtime thing. The string Manager is something which, I mean, right now is a constant but I cannot take the string and I cannot put in quotes Manager and make it a Manager. It has to be something generic. So, at run, at compile time I should have used a generic thing like object, but at runtime I can create specific instances of classes without knowing in advance which class it is going to be.

(Refer Slide Time: 12:00)

- From the `Class` object for class `C`, we can extract details about constructors, methods and fields of `C`
- Constructors, methods and fields themselves have structure
 - Constructors: arguments
 - Methods : arguments and return type
 - All three: modifiers `static`, `private` etc
- Additional classes `Constructor`, `Method`, `Field`
- Use `getConstructors()`, `getMethods()` and `getFields()` to obtain constructors, methods and fields of `C` in an array.



Madhavan Mukund

Reflection

Programming Concepts using Java 8 / 14

We can do still more with this class thing. So, a class, remember, consists of essentially three things. It consists of the constructors which are used when we create objects of the class, it consists of the instance variables or the fields which define the state of an object in this class and finally of course there are the methods which we use to probe or update values of these instance variables. So, for a given class, now what we can do in this reflection package in Java, is to extract these details about these constructors methods and fields.

So, of course, a constructor, a method and a field is more than just a name. So, they also have their own structures. So, if you look at a constructor, the constructor's name is fixed. Remember that a constructor in Java always has the same name as the class itself. However, it will have parameters in general. So, it will have argument. So, if I get a constructor I might want to know is there a constructor with this type of argument with that type of argument and so on.

If I have a method, well I have an argument but I also have a return type. Constructors do not return anything, they just implicitly create the object, but a method is something which takes an argument and potentially returns something. So, therefore I have arguments and a return type. And of course if I look at fields I have types associated with the fields. And finally, I have these modifiers like `private`, `static` and so on associated with all of them. So, all of these properties are there for all of these things, these modifiers.

So, in order to capture this information, I need more structure. It is not enough just to remember the name of a field or the name of a constructor as we said, we need to know more structures. So, therefore we actually have, now inside the reflection package we have other classes, so we have a class called Class with a capital C which describes a class as a whole, and inside the class we have constructors, methods and fields.

So, we have another set of classes defined in the reflecting package to capture properties. We have constructor class, we have a method class, we have a field class. So, for instance we can use functions like `getConstructors()`, `getMethods()` and `getFields()` to return the properties of the given class in an array of the appropriate type.

(Refer Slide Time: 14:21)



The class `Class` ...

- Extracting information about constructors, methods and fields

```
...  
Class c = obj.getClass();  
Constructor[] constructors = c.getConstructors();  
Method[] methods = c.getMethods();  
Field[] fields = c.getFields();  
...
```



- Extracting information about constructors, methods and fields

```
...
Class c = obj.getClass();
Constructor[] constructors = c.getConstructors();
Method[] methods = c.getMethods();
Field[] fields = c.getFields();
...
```

- `Constructor`, `Method`, `Field` in turn have functions to get further details




Madhavan Mukund

Reflection

Programming Concepts using Java 9/14

So, here is a typical thing. So, if I have a class, so `c` is a class, I have got an object, I have extracted its class in a variable called `c`. Now I take `c` and I ask what its constructors are. So, this will return to me in general an array or a list of constructors. So, the return type of this is an array of this constructor class. Similarly if I say `getMethods()`, I get an array of this method class, if I `getFields()`, I get an array of field class.

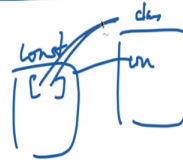
So, we have these structured classes or objects which allow us to extract the components of a given class. So, it is much more than just checking if two classes are the same and much more than knowing the name of the class. I can actually get the individual components. And now as we said each of these individual components has its own structure. So, they have their own properties. A constructor has arguments, each constructor in that array will have arguments, each method in the array will have a return value and argument and so on.

(Refer Slide Time: 15:25)

सिद्धिर्भवति कर्मजा

- Example: Get the list of parameters for each constructor

```
...
Class c = obj.getClass();
Constructor[] constructors = c.getConstructors();
for (int i = 0; i < constructors.length; i++){
    Class params[] = constructors[i].getParameterTypes();
    ...
}
```



Madhavan Mukund

Reflection

Programming Concepts using Java 10/14

- Example: Get the list of parameters for each constructor

```
...
Class c = obj.getClass();
Constructor[] constructors = c.getConstructors();
for (int i = 0; i < constructors.length; i++){
    Class params[] = constructors[i].getParameterTypes();
    ...
}
```

- Each parameter list is a list of types
 - Return value is an array of type `Class`



Madhavan Mukund

Reflection

Programming Concepts using Java 10/14

So for instance, I can get the parameters for each constructor. So, I have this constructor array which I got here, and now I can say for every *i* take constructors of *i* and get its parameter types. And this will now be what? It will be a list or an array of arguments. So, the signature of a constructor is given by the types of its arguments.

The actual names of the arguments are not relevant. You can use any name you want for an input parameter. But what you want to know is that this is a constructor which takes an `int` in the first position and a `String` in the second position. This is a constructor which takes no argument. So, the constructor which takes only a `String`, and so on. So, all these are different signatures and

each signature can be captured by the array of arguments. So, you can get the arguments and store it.

And now, what are these arguments? So, these represent types, and since they represent types, each of them is again a class. So, we had a class, and then we had this constructor here, and now we have a list of arguments which are all again classes. And then we can again go into that and so on. We can ask what happens, what is each of the arguments look like and so on.

So, each parameter list is a list of types and therefore the return value is also an array of type class. So, for each parameter I get an array of class, so I get in some sense, for a list of parameters, I get an array of classes. So, each constructor i gives me one array, and across all the i's, I get an array, if I store them all, I will get an array of such arrays.

(Refer Slide Time: 17:03)

IIT Madras BSc Degree

The class `Class` ...

- We can also invoke methods and examine/set values of fields.

```
...
Class c = obj.getClass();
..
Method[] methods = c.getMethods();
Object[] args = { ... }
// construct an array of arguments
methods[3].invoke(obj, args);
// invoke methods[3] on obj with arguments args
...
```

obj -> f(obj, ...)

f(obj, ...)

IIT Madras BSc Degree

Reflection

Programming Concepts using Java 11/14

So now again, in this inter session mode where we can intervene and do something, so one thing we said is given a class, we can launch it, we can create an object of that class. Now we can also change things in that class to suit our convenience. So, for instance, we can take a method, so we have got the set of methods, so we can take method at index 3, and we can invoke it. We can actually trigger it on that object, and we can pass the things that we want to do.

So, in this particular example, so we are not using, so we have this class, we have this object and then we have got its class, and from its class we have got the methods, and now we want to invoke method index 3 on that object, so we pass the object and the arguments to method 3, and say please invoke it. So, really we are doing, calling this method now like a function.

Normally we would say object dot f something. So, now we are saying f object that something. So, we are passing it, because we are not operating on the object from inside as we would when we are programming it, but from outside because we are reflecting on this data.

(Refer Slide Time: 18:35)

IIT Madras
BSc Degree

The class `Class` ...

- We can also invoke methods and examine/set values of fields.

```
...
Class c = obj.getClass();
..
Method[] methods = c.getMethods();
Object[] args = { ... }
// construct an array of arguments
methods[3].invoke(obj,args);
// invoke methods[3] on obj with arguments args
...

Field[] fields = c.getFields();
Object o = fields[2].get(obj);
// get the value of fields[2] from obj
...
fields[3].set(obj,value);
// set the value of fields[3] in obj to value
...
```

an Mukund Reflection Programming Concepts using Java 11/14

In the same way we can also take a field of an object and assign it. So, we can get the fields of this thing and then we can maybe get some information about one particular field and then I can take another field in this object and set it to the value of, so I can copy the value which is there in field number 2 into field number 3. So, these are all these ways of interceding.

I can start with a class and create an object, I can take that object, and I can apply a function to it, I can take the object I can change the value of an instance variable. So, sitting within Java I can actually manipulate a Java program as it is running.

(Refer Slide Time: 19:13)



Reflection and security

- Can we extract information about private methods, fields, ...?
- `getConstructors()`, ... only return publicly defined values
- Separate functions to also include private components
 - `getDeclaredConstructors()`
 - `getDeclaredMethods()`
 - `getDeclaredFields()`
- Should this be allowed to all programs?
- Security issue!
- Access to private components may be restricted through external security policies



So, all of this works at what level? So, does it allow us to look only at public information or at private information also? So, we know that if we are actually programming in Java, then a class can only access public information about another class. It cannot access private information. Even a sub class cannot look at private information about super class and this is the way of ensuring some encapsulation discipline across programmers. So, people who are coding cannot violate this discipline normally.

So, it turns out that these functions that we talked about only return the publicly defined values but there are functions in the `reflect` package which give us private values because this reflection package is not really intended to be used to develop an application. It is actually more as a way of building things like debuggers and testing kits. So, you want to check whether in the course of the program running it is doing the thing.

So, this is more like a higher level thing. So, you have a piece of code which you are supposed to design to do a particular task, and now you want to be able to also validate that code is doing the thing by asking questions about what is happening to the code as its running. So, from that perspective, this is, reflection is to be used by the person who has actually designed the code.

And therefore, they have access to the private components by using these more elaborate methods called `getDeclaredConstructors()`, `getDeclaredMethods()` and `getDeclaredFields()` . So, this includes the public and the private, whereas the other one had only the public fields.

So as I said, this is intended to be used by the designer of the code and therefore, it is more intended to use for internal validation. But can it be allowed to all programs? And it turns out that Java does not really prohibit this. So, if you use reflection, you can take arbitrary code and you can actually pass things around. If you can get access to something you can inspect it in some sense. You can do a Java inspection from within Java using reflection.

So, there is some amount of security involved because if this Java code is for instance, something which is running on a web page or something then a user of the web page can actually inspect what is happening inside the code and perhaps take some malicious action or do something based on that. So, there are situations where you can actually define a security policy which is external to the Java code itself.

So, it is not something that you can, I cannot write in my class definition do not expose this private. That is not allowed. So, I cannot really easily take a piece of Java code and say within this code do not allow reflection package to look at this at all, no matter who asks for it. That is not possible.

But you can write an external thing saying that in the, because remember the Java programs run in this JVM kind of thing, so they run in a controlled environment. So, you can say that in this JVM environment in which the Java code is running, this is not allowed. These fields cannot be exposed. So, that way you can control access, but Java itself internally cannot prohibit somebody using reflection from looking up private values.

(Refer Slide Time: 22:17)



Using reflection

- BlueJ, a programming environment to learn Java
- Can define and compile Java classes
- For compiled code, create object, invoke methods, examine state
- Uses reflective capabilities of Java — BlueJ need not internally maintain “debugging” information about each class
- See <http://www.bluej.org>



Madhavan Mukund

Reflection

Programming Concepts using Java 13/14

So, what might be a typical use case for this kind of thing? So there is an interesting project which is called BlueJ, which is a programming environment to teach Java to small kids. So, it is an interactive kind of IDE, in which you can develop individual classes, you do not have to write a very full scale elaborate program, the skeleton and the main and all that.

So, if you just want to define a class which does certain things with certain methods then you can define it and there will be a kind of automatic way in which it will be plugged in to something that can execute. So, this is called BlueJ.

So, you can define and compile Java classes and then you can run it like a debugger. You can create, you can click on it, create object so if you have used some of these programming things like scratch and all that, this is a bit like that but this is at the Java level. You can create an object then you can invoke a method on that object. So, it is a bit like using an interactive debugger but is more intended for pedagogical reasons, for teaching Java. So, you can create an object, you can say what happens if I call this function with these parameters, what is the current state of all the values and so on.

So, you can imagine that the way that such a package would be written would be to use reflection. So, BlueJ internally is allowing the user to interact with the program like this by looking up what I, because remember this is a general package, so BlueJ will take arbitrary Java

code which somebody has written and it will allow them to poke it, run methods on it change values and so on. So, clearly this kind of introspection and this reflection is being used inside BlueJ for this.

So, this is one use case. Rather than debugging your own program, you can provide this kind of a wrapper environment where people can create Java code and experiment with it without going through the whole machinery of writing their own test code and so on. So, you can look at this url www.bluej.org to see more about BlueJ.

(Refer Slide Time: 24:21)



Limitations of Java reflection

- Cannot create or modify classes at run time
 - The following is not possible

```
Class c = new Class(...);
```
 - An environment like BlueJ must invoke Java compiler before you can use a new class
- Contrast with Python
 - `class XYZ:` can be executed at runtime in Python
- Other OO languages like Smalltalk allow redefining methods at run time



Madhavan Mukund

Reflection

Programming Concepts using Java

14 / 14

So, one of the things that you cannot do is to create new classes. So, what we can do in Java reflection is to take the existing objects that are there and get information about the class and then do various things with that. We can get information about the constructors, the methods, the fields, we can apply, we can create new objects, so we can implicitly call the constructor, we can explicitly run a method, we can explicitly update a field, and so on.

But if I want to create a new class, for example in BlueJ, I can take a class which I have written and then start playing with it, but if I want to add a class then BlueJ will have to kind of use the Java compiler to include that class. BlueJ on its own, while it is running, cannot incorporate a new class into the Java program. So, you cannot, at runtime, generate a new class. You can

generate objects of a class but not the class itself. The class itself has to go through Java compiler which will then check various things about it.

So in contrast, in Python if we use a class definition, it is an executable piece of code. So, anywhere in my code I can, if I run across a class definition it will create a class. So, classes can be created at runtime in a language like Python. So, this is not possible with Java. Other languages also, so there are, there is a very, we talked about Simula as one of the pioneers in object oriented programming, but the next big language after Simula was Smalltalk.

So, actually Java inherits a lot of features and a lot of experience from these languages. So, Smalltalk was a very pioneering object oriented language which was developed in the 1970s at Xerox PARC. And in similar, for example, you can do this, I mean, in Smalltalk you can do this kind of runtime modification of your code. You can create methods and so on, very much like what Python can do.

So, Java reflection can do a lot of things but it is not universal. It cannot do everything. But it is very useful in order to be able to do these kind of introspective programming to find out what is happening in your code and to modify it and make changes and experiment with it, and especially for debugging typically, but also for these languages like, this package like BlueJ. To teach Java, you can use it.